

Concrete Strength Prediction Using Machine Learning

1. Introduction

The goal of this project is to build a machine learning model to predict the maximum load capacity of concrete (**Max_Load_lbf**) using a reduced set of meaningful features. Predicting concrete strength is important in civil engineering because it helps ensure safety, optimize material usage, and reduce construction costs.

Instead of using all available variables, this project focuses on selecting a small number of important features. This makes the model simpler, faster, and easier to understand while still maintaining strong prediction performance.

This work is inspired by existing research but builds a custom prototype using selected features and multiple machine learning models.

2. Dataset Description

The dataset contains **178 samples** of concrete mixtures. Each sample includes information about material composition, ratios, and physical properties.

The **target variable** is:

- **Max_Load_lbf** → Maximum load the concrete can carry)

The dataset includes features such as:

- Age of concrete
- Cement content
- Water content
- Material ratios
- Density and weight

3. Feature Engineering and Selection

Instead of using all features, we selected **5 important features** based on domain knowledge and experimentation:

- **Log_Age** → captures curing effect of concrete
- **WaterCement_Ratio** → key factor affecting strength
- **PCC_Cement_Ratio** → relationship between PCC and cement
- **Density_lb_per_in3** → physical density of material
- **Weight_lb** → total weight contribution

Additional features were created but not used in Version 1:

- Age_squared
- Age_x_WC

The goal was to keep the model simple while maintaining good accuracy.

4. Models Used

We trained and compared **5 regression models**:

- Linear Regression
- Ridge Regression
- ElasticNet
- Random Forest Regressor
- XGBoost Regressor

These models include:

- **Simple linear models** → easier to interpret
- **Complex ensemble models** → more powerful but risk overfitting

5. Methodology

The following steps were performed:

1. Load dataset from Excel file
2. Clean data (remove missing target values)
3. Create engineered features
4. Select 5 final features
5. Split dataset (80% train, 20% test)
6. Train models using pipelines
7. Evaluate using:
 - R^2 score (accuracy)
 - MAE (Mean Absolute Error)
 - RMSE (Root Mean Squared Error)
8. Perform **10-fold Cross Validation**
9. Compare models and select the best
10. Save best model and visualization

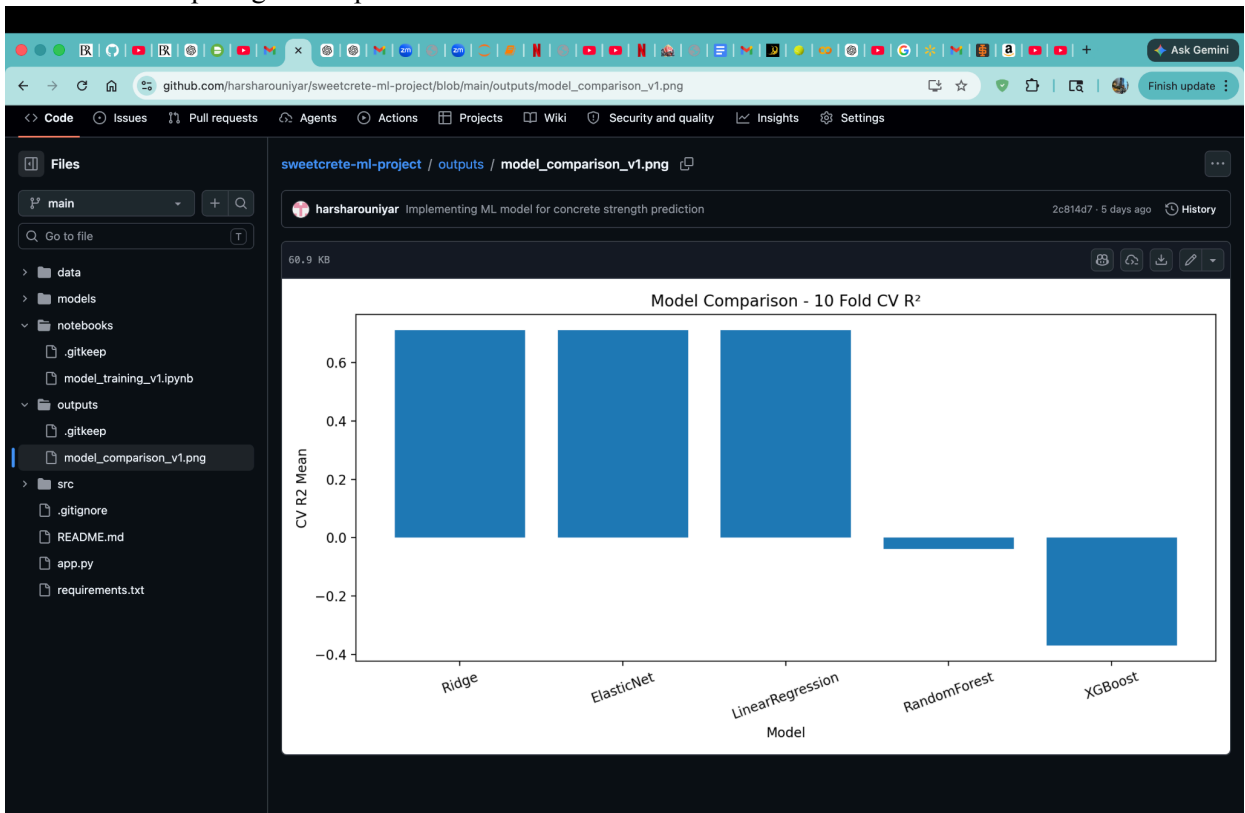
6. Results

Model Performance Table

Model	Test R ²	MAE	RMSE	CV R ²	Mean CV R ²	STD	Ridge	0.784875	8391.27	11657.90
0.711165	0.275276	ElasticNet	0.785074	8390.10	11652.49	0.711121	0.275523	Linear Regression		
0.785543	8387.23	11639.78	0.711006	0.275960	Random Forest	0.758735	7277.52	12345.86		
-0.038675	1.546612	XGBoost	0.585543	8327.76	16181.34	-0.368387	1.920727			

Visualization

A bar chart comparing model performance



7. Analysis of Results

The results show that:

- **Linear models performed best**
- Ridge, ElasticNet, and Linear Regression all achieved similar performance (~ 0.71 CV R^2)
- Random Forest and XGBoost performed poorly in cross-validation

Why this happened:

- The dataset is **small (178 samples)**
- Complex models **overfit small datasets**
- The relationship between features and target is mostly **linear**
- Regularization (Ridge, ElasticNet) helps improve stability

Even though Random Forest had lower MAE, its **negative CV score shows poor generalization**, which is more important.

8. Final Model Selection

The final selected model is:

Ridge Regression

Why Ridge?

- Highest and most stable cross-validation score
- Strong test performance
- Works well with small datasets
- Reduces overfitting using regularization

9. Output of the Project

Files Created

- **Trained Model:**
`models/best_model_v1.pkl`
- **Performance Graph:**
`outputs/model_comparison_v1.png`
- **Notebook:**
`notebooks/model_training_v1.ipynb`

10. GUI Implementation

A **Graphical User Interface (GUI)** was added to make the model easy to use.

Instead of running Python code, users can directly input values and get predictions.

User Inputs in GUI:

- Log_Age
- WaterCement_Ratio
- PCC_Cement_Ratio
- Density_lb_per_in3
- Weight_lb

Output:

- Predicted **Max_Load_lbf**

Why GUI is important:

- Makes the project practical
- Easy for non-technical users
- Turns model into real application
- No need to run code manually

11. Conclusion

This project successfully built a machine learning model to predict concrete maximum load using a reduced feature set.

Key Conclusions:

- A small number of well-selected features can produce strong predictions •
- Linear models are more effective than complex models for small datasets •
- Ridge Regression provided the most reliable performance
- GUI makes the model easy to use in real-world scenarios

Git hub link <https://github.com/harsharouniyar/sweetcrete-ml-project/tree/main/>